

Руководство по кодированию файла

Специальная монография по прямой половине кодека DataShield — преобразованию файла в поток пакетов FEC. Документ дополняет [AlgorithmGuide.ru.md](#) (разделы 2–4 дают обзор): здесь та же тема разобрана формально — постановка задачи, определения и инварианты, пошаговые алгоритмы с обоснованиями, оценки объёма выхода, проработанные примеры и точные границы гарантий. Инженерный пересказ с совпадающей нумерацией разделов — [EncoderGuide.Engineer.ru.md](#); вводное изложение «с нуля» — [EncoderGuide.Tutorial.ru.md](#). Обратное преобразование — семейство [DecoderGuide.*](#); арбитраж результата — семейство [AssemblyGuide.*](#).

1. Постановка задачи

Пусть заданы содержимое $F \in \{0,1\}^*$, $|F| = S$, и имя `name`. Требуется построить упорядоченную последовательность пакетов $P = (P_1, \dots, P_L)$, $|P_i| = 75$, удовлетворяющую требованиям:

- Декодируемость.** Из любого достаточно «выжившего» подмножества P декодер восстанавливает F бит-в-бит (доказательство корректности — совпадение SHA-256, см. [AssemblyGuide.Academic.ru.md](#)).
- Самопроверяемость.** Каждый пакет проверяем либо автономно (заголовок), либо при знании заголовка своего файла (сектор).
- Отказ вместо брака.** Любое нарушение ограничений завершается исключением; частичный выход невозможен.

Параметры избыточности: $\text{eccPercent } p \in [0, 100]$, $\text{headerPercent } h \in [0, 100]$; умолчания 10 и 3.

2. Модель потерь и избыточность

Модель повреждений канала: (а) стирание интервалов потока; (б) вставка мусора и рассинхронизация границ; (в) целенаправленная подмена пакетов. Ответы: (а) — стирающий код Рида–Соломона и повторение заголовков; (б) — скользящее окно декодера с шагом 1 (см. [DecoderGuide.Academic.en/ru.md](#), разделы 5–6); (в) — привязка секторов сидом `H5` и финальный арбитраж `H3`.

Определения размерностей:

```
N = max(1, [S / 64])           (data-тома)
M = 0, если p = 0; иначе max(1, [N·p / 100])  (ECC-тома)
T = N + M                       (секторов данных)
H = max(3, [T·h / 100])         (копий заголовка)
L = T + H                       (пакетов в потоке)
```

Все округления — вверх; $\max(1, \cdot)$ гарантирует нетривиальность операций кодирования и опознавания.

3. Формат: константы и структуры

Пакет — 75 байт. Заголовок: $H1 \parallel H2 \parallel H3 \parallel H4 \parallel H5$, где $H1$ — имя (14 байт, ASCII), $H2$ — размер S (3 байта LE, $S \leq 2^{24}-1$), $H3$ — SHA-256 содержимого (32), $H4$ — M (2 байта LE), $H5$ — 24 байта. Сектор: $D1 \parallel D2 \parallel D3$, где $D1$ — номер (2 байта LE), $D2$ — payload (64), $D3$ — 9 байт. Потолок $T \leq 2^{16}-1$ обусловлен размером поля $GF(2^{16})$ кода Рида–Соломона.

Инвариант I1 (непротиворечивость размерностей). Заголовок однозначно определяет N , M , T по S и $H4$; кодер и декодер вычисляют их одинаковыми формулами, поэтому несогласованные размерности невозможны.

4. Подготовка данных

Шаги: валидация $S \leq 2^{24}-1$; упаковка имени $H1 = \text{Pack}(\text{name})$; $H3 = \text{SHA-256}(F)$; вычисление N , M , T с проверкой $T \leq 2^{16}-1$; нарезка F на payload $d_0, \dots, d_{N-1}$ (последний добивается нулями: $d_{N-1} = F[(N-1) \cdot 64 .. S] \parallel 0^{N \cdot 64 - S}$).

Предложение 4.1 (инъективность нарезки). Отображение $F \mapsto (d_0, \dots, d_{N-1}, S)$ инъективно: S известно из $H2$, поэтому $F = [d_0 \parallel \dots \parallel d_{N-1}]_{\{0..S\}}$ восстанавливается отбрасыванием хвостовой добивки. ■

Упаковка имени: имя разбивается по первому вхождению точки; суффикс-расширение сохраняется полностью; база усекается до бюджета с маркером $\sim$; при бюджете базы < 2 — исключение (имя нерепрезентабимо). Полное имя не передаётся: $H1$ служит человекочитаемой меткой.

5. ЕСС-кодирование

Используется стирающий код Рида–Соломона над $GF(2^{16})$. Payload тома — 32 символа поля (UInt16 LE). Для каждой позиции $s \in [0, 32)$ независимо: столбец $c = (d_0[s], \dots, d_{N-1}[s]) \in GF(2^{16})^N$ линейным оператором `RsRaid16.Process` продолжается до (c, e) с $e \in GF(2^{16})^M$; ЕСС-том j получает символ $e_j[s]$ в позиции s .

Предложение 5.1 (условие восстановления). Пусть доступно множество $A \subseteq [0, T)$ принятых томов с $|[0, N) \cap A| = N - k$ (потеряно k data) и $|[N, T) \cap A| \geq k$. Тогда все data-тома восстанавливаются точно. Доказательство: схема Рида–Соломона — систематический MDS-код с расстоянием $M+1$; любые N из $N+M$ компонент кодового слова определяют слово однозначно. Стирания (позиции потерь известны из карты) исключают неоднозначность декодирования. ■

Следствие: потерянные ЕСС-тома на восстановление не влияют. Граничное условие $N + M \leq 2^{16} - 1$ — размер поля.

Инвариант I2 (детерминированность ЕСС). Кодирование — чистая функция от $(d_0, \dots, d_{N-1}, N, M)$: одинаковый вход даёт одинаковые ЕСС-тома (в рамках фиксированного эталонного `RsRaid16`).

6. Заголовочный пакет и H5

$H5 = \text{Trunc}_{24}(\text{SHA-256}(H1 \parallel H2 \parallel H3 \parallel H4))$. Заголовочный пакет — $H1 \parallel H2 \parallel H3 \parallel H4 \parallel H5$.

Предложение 6.1 (автономная проверяемость). Корректность заголовка проверяется без внешних данных: $\text{Trunc}_{24}(\text{SHA-256}(\text{первые 51 байт})) = \text{последние 24 байта}$. Вероятность ложного принятия случайного пакета — 2^{-192} . ■

Предложение 6.2 (сидовая привязка). Знание $H5$ необходимо для проверки любого сектора (раздел 7); $H5$ не выводится из секторов за разумное время (стойкость SHA-256). Следовательно, множество секторов файла образует хеш-связанную структуру с корнем в заголовке. ■

7. Секторы данных и D3

Для $i \in [0, T)$: $D1 = \text{LE16}(i)$, $D2 = \text{payload}_i$, $D3 = \text{Trunc}_9(\text{SHA-256}(H5 \parallel D1 \parallel D2))$ (вход — 90 байт).

Предложение 7.1 (устойчивость к склейке). Вероятность прохождения проверки случайного/чужого пакета — 2^{-72} на пакет; для потока из L пакетов — $\approx L \cdot 2^{-72}$. Практически недостижимо; целенаправленная подмена требует прообраза усечённого SHA-256 с известным сидом. ■

Инвариант I3 (полнота покрытия). Payload-и секторов $0..T-1$ — это в точности $d_0 \dots d_{N-1}$ и ЕСС-тома; никакой payload не теряется и не дублируется при сборке потока.

8. Расстановка заголовков

`ArrangePackets`: $P_1 = \text{header}$; $P_{L+1} = \text{header}$; промежуточные копии вставляются после каждого $\lceil T/(H-1) \rceil$ -го сектора ($\text{interval} = \max(1, T/(H-2+1))$), вставка при $(i+1) \bmod \text{interval} = 0$, пока остались копии).

Предложение 8.1 (равномерность). Расстояние между соседними копиями заголовка в потоке — $\Theta(T/H)$ с точностью до константы; вырезание любого связного интервала длины $< \text{interval} - 1$ секторов оставляет неповреждённой хотя бы одну копию. ■ (Границы интервала — следствие конструкции вставки.)

9. Выходные форматы и PacketIO

Binary: конкатенация пакетов, $75 \cdot L$ байт. Base64: `Convert.ToBase64String(Pi)` на строку; так как $75 \equiv 0 \pmod{3}$, длина строки ровно 100, символов добавки нет.

Предложение 9.1 (оценка расширения). Binary: $75(T+H)/S$; при $S \rightarrow \infty$, p, h фиксированных: $75/64 \cdot (1 + p/100)(1 + h/100) + o(1)$ (для $p = h$ типовых $10/3: \approx 1.326$). Base64: умножить на $4/3$ (≈ 1.768) плюс по одному переводу строки на пакет. ■

Декоративные обрамления $>[...][...][SHA-256:...]/<[...]$ не входят в полезную нагрузку и игнорируются декодером (фильтр отбрасывает не-Base64 байты).

10. Прогресс и отмена

Фазы глобальной шкалы: подготовка $[0, 10)$, ECC $[10, 75)$ (линейно масштабируется по 32 символам через `ScaledProgress`), пакетизация $[75, 100)$, терминальный отчёт `Done` (100). Троттлинг `ProgressThrottle` по целым процентам. `CancellationToken` проверяется на границах фаз и внутри символьного цикла ECC; отмена выбрасывает `OperationCanceledException` — частичный результат не формируется.

Инвариант I4 (атомарность). `Encode` либо возвращает полный список `L` пакетов, либо бросает исключение; промежуточные состояния наружу не наблюдаются.

11. Статистика и гигиена памяти

`EncodeWithStats` возвращает `EncodeStats(S, H3, N, M, L, H)`. После успешной сборки обнуляются: все data-payload, все ECC-payload, сериализованный заголовок.

Инвариант I5 (минимизация остатка). По завершении `Encode` в куче остаются только: возвращённые пакеты и буферы, недостижимые для пользователя. Число копий содержимого `F` в памяти — по числу пакетов-секторов (неизбежно) плюс ноль лишних.

12. Поточковый вход

`ReadStreamContent`: для позиционируемого потока — чтение остатка $Length - Position$ с предварительной проверкой потолка; для непозиционируемого — копия через `MemoryStream` с пост-проверкой. Поток не закрывается вызывающей стороной кодера. Полное чтение обязательно: и `H3`, и RS требуют целого `F`.

13. Проработанные примеры

13.1. Файл 1000 байт, $p=10$, $h=3$

$N=16, M=2, T=18, H=3, L=20$; интервал вставки $\lceil 18/2 \rceil = 9$; поток $H D_0 \dots D_8 H D_9 \dots D_{17} H$. Binary 1500 байт (150%), Base64 2000 символов + 20 переводов строки.

13.2. Пустой файл

$S=0 \Rightarrow N=1$: единственный data-том — 64 нуля; $M \geq 1$ при $p \geq 1$. Файл существует, собирается, $H3$
= SHA-256($\emptyset$) .

13.3. Однобайтовый файл, $p=10$

$N=1, M=1, T=2, H=3, L=5$. Избыточность 6400% — цена нижней границы формулы $M \geq 1$.

13.4. Предельный файл

$S = 16\,777\,215$: $N = 262\,145 > 65\,535$ даже при $M=0$ — исключение по потолку $GF(2^{16})$.
Отсюда фактический потолок при $p=0$: $S \leq 65\,535 \cdot 64 = 4\,194\,240$ байт (4 МиБ минус 256 байт); с ECC — меньше.

13.5. Имя `my.documents.2024.xlsx`

Разрез по первой точке: база `my` , расширение `.documents.2024.xlsx` (20 байт) > 12 — бюджет базы отрицателен: исключение «не представимо».

14. Параметры и пределы

Параметр	Область	Следствие нарушения
<code>eccPercent</code>	$[0, 100]$ (фактически — пока $T \leq 65\,535$)	<code>ArgumentOutOfRangeException</code> при < 0
<code>headerPercent</code>	$[0, 100]$	<code>ArgumentOutOfRangeException</code> при < 0
S	$\leq 16\,777\,215$	<code>InvalidOperationException</code>
$T = N+M$	$\leq 65\,535$	<code>InvalidOperationException</code>
Имя	база ≥ 2 байт после резерва расширения	<code>InvalidOperationException</code>

15. Краевые случаи

Сведены в разделы 13.2–13.5 и таблицу раздела 14. Дополнительно: $p=0$ пропускает ECC-фазу целиком (нулевой overhead по времени и памяти); непозиционируемый вход допускается, но требует промежуточной копии в памяти.

16. Сводка инвариантов и гарантий

Инварианты: **I1** согласованность размерностей; **I2** детерминированность ECC; **I3** полнота покрытия payload; **I4** атомарность Encode; **I5** минимизация остатка памяти.

Гарантии:

1. Полный корректный поток либо исключение (I4).
2. $H3$ — точный SHA-256 содержимого; арбитраж сборки безупречен (Предложение 4.1).
3. Заголовок проверяем автономно с ложным срабатыванием 2^{-192} (6.1); секторы — только через $H5$ с 2^{-72} (7.1).
4. Потеря $\leq$ доступных ECC data-томов восстановима точно (5.1).
5. Минимум 3 копии заголовка с равномерными интервалами $\Theta(T/H)$ (8.1).
6. Оценка расширения: $75/64 \cdot (1+p/100) (1+h/100)$ binary; $\times 4/3$ Base64 (9.1).

Числовая сводка: пакет 75 байт; payload 64; $H5$ 192 бита; $D3$ 72 бита; $S \leq 16\,777\,215$; $T \leq 65\,535$; расширение при (10,3) $\approx 132\%/176\%$.